M.Tech (Cloud Computing) - 3rd Semester Project

PROGRESS REPORT

Cost-Aware Predictive Resource Orchestration for Distributed Cloud Workloads Using Workload Forecasting

Submitted by	Aman Kumar (Roll no. PA2507MTE08)
Supervisor	Sunil Chauhan, Technical Engineering Manager & Architect Netcracker Technology
Institute	Indian Institute of Technology Patna
Reporting period	3rd semester, 25th November 2026
Prototype version	cloud-orchestration-sim 0.1.0-SNAPSHOT (Java 17, Maven)

Status of this document: interim progress report on a research prototype. All results are from a discrete-event simulation on synthetic workloads; they have not been validated on a real cloud platform.

Contents

Abstract 3

1. Introduction and Problem Statement 3

1.1 Objectives . . . 3

2. Summary of Progress 4

3. System Design 4

3.1 Architecture . . . 4

3.2 Simulation model . . . 5

3.3 Synthetic workload . . . 5

3.4 Forecasters (Module A and predictive policies) . . . 5

3.5 Scaling policies (Module B) . . . 5

3.6 Cost model and metrics . . . 5

4. Implementation 6

5. Experimental Setup 6

6. Results to Date 7

6.1 Module A - workload forecasting . . . 7

6.2 Module B - scaling policies (default configuration) . . . 8

6.3 Effect of billing granularity . . . 9

6.4 Sensitivity to cold-start delay . . . 9

7. Verification Status 10

8. Assumptions and Limitations 10

8.1 Assumptions . . . 10

8.2 Limitations and threats to validity . . . 10

9. Demonstration Readiness 11

10. Plan for the Remaining Work (proposed) 11

References 11

Abstract

Elastic cloud platforms let an operator change the number of worker instances as demand changes, but every decision trades cost against latency: too few workers and tasks queue, too many and paid capacity sits idle. Reactive autoscalers act only after load has changed, so new capacity arrives one control interval plus one cold-start delay late. This project studies whether forecasting the workload, and making scale-in decisions aware of how the provider bills, gives a better cost/latency trade-off. During this semester a Java 17 research prototype was built: a shared, deterministic discrete-event simulator of a cloud worker pool, plus two independently executable research modules - (A) evaluation of moving-average workload forecasters and (B) comparison of fixed, reactive, predictive and cost-aware predictive scaling policies. Experiments over five random seeds on synthetic workloads show that, in these settings, predictive scaling used about 31 % fewer worker-hours than the reactive baseline with a lower mean p95 latency but a higher SLA-violation rate; that billing-boundary-aware scale-in made no difference under per-minute billing but cut cost by about 45 % under per-hour billing; and that flat moving-average forecasts degrade sharply when cold starts are long. These are simulation results with small sample sizes and are reported as preliminary.

1. Introduction and Problem Statement

Distributed cloud workloads such as batch jobs, request processing and data pipelines are served by pools of virtual machines or containers. Demand varies with daily cycles and with short, unpredictable bursts. A resource orchestrator must decide, at regular intervals, how many workers to run. Three facts make this hard:

- **Cold starts.** A newly requested worker takes tens of seconds to minutes before it can accept work.
- **Billing granularity.** Providers charge in blocks (per second with a minimum, per minute, or per hour). Releasing a worker early in a paid block saves nothing, and re-launching it starts a new block.
- **Tail latency.** Users experience the slowest requests, so the 95th-percentile latency matters more than the mean.

Problem statement. Given a time-varying stream of tasks, choose the worker-pool size at each control interval to minimise simulated cost while keeping queue wait and p95 latency low, using only monitoring data available at decision time.

1.1 Objectives

- O1 - Build a reproducible, cost-free simulation framework for a configurable worker pool with synthetic workloads.
- O2 - Implement and evaluate moving-average workload forecasters (Module A).
- O3 - Implement fixed, reactive and forecast-driven predictive scaling, and a billing-aware variant (Module B).
- O4 - Measure queue wait, p95 latency, throughput, worker-hours, simulated cost and forecast error; export CSV.
- O5 - Document assumptions and limitations honestly, and prepare the prototype for demonstration.

2. Summary of Progress

Objective / deliverable	Status	Evidence
O1 Shared simulation framework	Done (prototype)	framework package, 2,316 lines; 12 engine tests
O2 Module A: forecasting	Done (prototype)	NAIVE, SMA, WMA, EMA; walk-forward MAE/RMSE/MAPE
O3 Module B: scaling policies	Done (prototype)	Fixed, Reactive, Predictive, Predictive-CA
O4 Metrics and CSV export	Done	summary, aggregate and per-interval CSVs
Reproducibility (fixed seeds)	Done	identical CSVs on re-run (checked)
Unit tests	Done, with caveat	66 tests pass; run via javac (see Section 7)
Maven build (mvn clean verify)	Pending	blocked by network policy in dev sandbox
Validation on real cloud / real traces	Not started	planned (Section 10)
Trend-aware forecasting, cost-aware sizing	Not started	planned (Section 10)

Code base at the time of writing: 41 main source files (3,076 lines) and 13 test files (1,075 lines); no runtime dependencies beyond the JDK; JUnit 5 for tests.

3. System Design

3.1 Architecture

The application is split into one shared framework and two research modules. The framework knows nothing about the modules; the modules depend only on the framework and never on each other. An automated test (ModuleIndependenceTest) enforces both rules on every build.

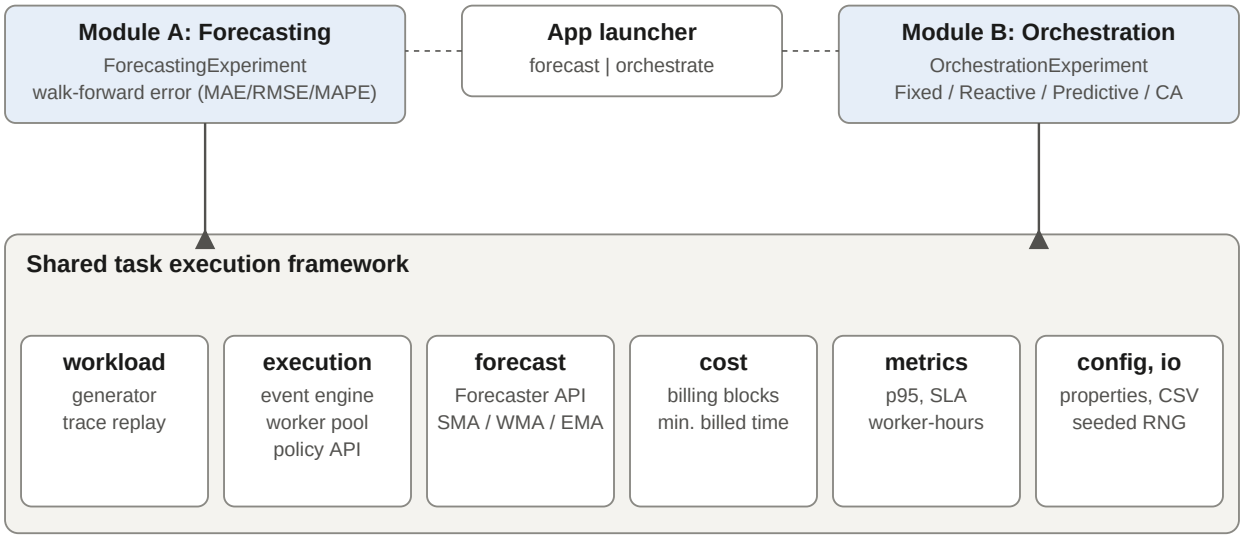


Figure 1. Architecture. Solid arrows: compile-time dependencies. Dashed: the launcher only dispatches to either module's main class.

3.2 Simulation model

The core is a deterministic discrete-event simulator. Events (task arrival, task completion, worker ready, scheduled termination, control tick) are processed in time order. Tasks wait in one FIFO queue and each worker runs one task at a time. Every control interval (60 s by default) the scaling policy receives only what a real monitoring system would provide - arrivals and completions in the last interval, queue length, worker counts, utilization and the observed mean service time - and returns a target pool size. Scale-out workers become ready after a configurable cold-start delay; scale-in cancels provisioning workers first, then idle workers, then lets busy workers drain. Simulated time is independent of wall-clock time, so a run is exactly repeatable.

3.3 Synthetic workload

Arrivals form a non-homogeneous Poisson process generated by Lewis-Shedler thinning [3], with rate

```
rate(t) = base x (1 + trend x t/3600) x (1 + A x sin(2 pi t / T + phase)) x
burstFactor(t)
```

where random bursts start at each minute with probability burstsPerHour/60. Service times are log-normal (mean 2 s, coefficient of variation 0.5) in the sample configurations. Each seed derives independent named random streams for arrivals, bursts and service times; within a seed, every policy is replayed on the identical task list (common random numbers), so policy differences are not caused by different workloads. Per-interval counts can alternatively be loaded from a CSV trace.

3.4 Forecasters (Module A and predictive policies)

NAIVE (last value), simple moving average SMA(w), linearly weighted moving average WMA(w) and exponential moving average EMA(alpha) [5]. All produce a flat forecast for every horizon. Module A evaluates them walk-forward: the forecast for interval k uses only intervals before k, and a warm-up equal to the largest window plus horizon is excluded so all models are scored on the same points.

3.5 Scaling policies (Module B)

- **Fixed-n**: constant n workers (12 = sized near mean load, 32 = sized near burst peak).
- **Reactive**: threshold rules - add 4 workers when utilization is at least 0.80 or there are at least 2 queued tasks per worker; remove 2 when utilization is at most 0.40 and the queue is empty; cooldowns of 1 and 3 intervals.
- **Predictive**: forecast the next interval's arrivals with SMA(3) and size the pool using Little's law [4]:

```
required = ceil( lambda*S / u_target * (1 + margin) + queue*S / (interval *
drainIntervals) )
```

with lambda the forecast arrival rate, S the observed mean service time, u_target = 0.75 and margin = 0.10. Scale-in is held for two intervals after any increase.

- **Predictive-CA (cost-aware)**: same sizing, but a worker chosen for removal keeps serving until the end of its already-paid billing block and is then released, unless a later scale-out reclaims it first.

3.6 Cost model and metrics

Each worker is billed from launch (including cold start) to termination, rounded up to whole billing blocks with a minimum billed time. Prices are abstract units (0.10 per worker-hour), not a provider's price list. Reported metrics: queue wait (start - arrival), latency (completion - arrival), nearest-rank p50/p95/p99, SLA-violation % (latency above 10 s, plus incomplete tasks), throughput, worker-hours, billed cost, utilization, and one-step-ahead forecast MAE/RMSE/MAPE.

4. Implementation

Package	Responsibility	Key classes
framework.execution	Discrete-event engine, worker pool, policy API	SimulationEngine, ScalingPolicy, ControlObservation
framework.workload	Synthetic generation, trace replay	WorkloadGenerator, WorkloadProfile, TraceLoader
framework.forecast	Forecasters and error metrics	Forecaster, SMA/WMA/EMA/Naive, ForecastErrorTracker
framework.cost / metrics	Billing, percentiles, aggregation	CostModel, SummaryMetrics, MetricAggregator
framework.config / io / random	Config files, CSV, seeded RNG	ExperimentConfig, CsvWriter, RandomSource
modules.forecasting	Module A experiment	ForecastingExperiment, ForecastingMain
modules.orchestration	Module B experiment and policies	OrchestrationExperiment, PolicyFactory, 3 policy classes

Technology choices: Java 17 (developed and run on OpenJDK 21), Maven with pinned plugin versions, JUnit 5. Spring Boot was considered and deliberately not used: the prototype is an offline batch simulator and would gain no feature from it. Experiments are driven by .properties files, with key=value command-line overrides for parameter sweeps. Both modules are runnable from one jar:

```
java -jar target/cloud-orchestration-sim-0.1.0-SNAPSHOT.jar forecast
configs/forecast-default.properties
java -jar target/cloud-orchestration-sim-0.1.0-SNAPSHOT.jar orchestrate
configs/orchestration-default.properties
```

5. Experimental Setup

Configuration	Workload	Pool and timing	Billing
forecast-default (A)	24 h; base 4 tasks/s; daily cycle ±60 %; ~1 burst/h (x2, 10 min)	5-min intervals; horizons 1 and 3	-
orchestration-default (B)	2 h; base 4 tasks/s; 1-h cycle ±60 %; 2 bursts/h (x2.5, 5 min)	1-40 workers; 60 s cold start; 60 s control	60 s blocks
orchestration-hourly (B)	same as default	same as default	1-hour blocks
orchestration-slow-prov. (B)	as default, 4 bursts/h	180 s cold start	60 s blocks

Every configuration was run with seeds 42-46 (five independent workloads). Tables report mean and sample standard deviation across seeds. One run of all four configurations takes about 4 s of wall-clock time.

6. Results to Date

6.1 Module A - workload forecasting

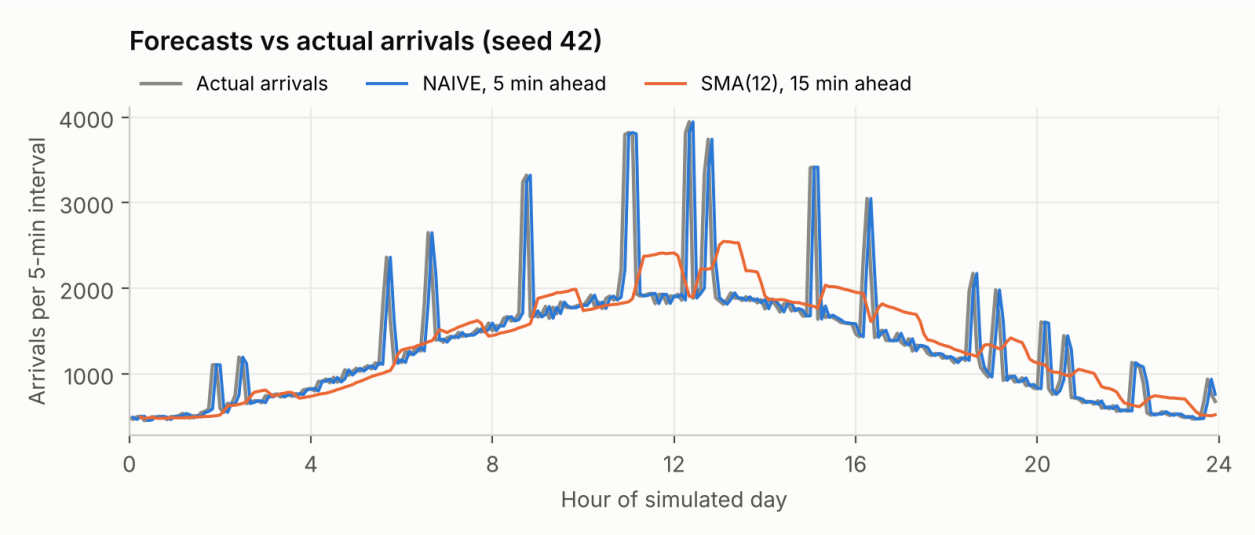


Figure 2. Actual arrivals per 5-minute interval for one simulated day (seed 42), with the 1-step NAIVE forecast and the 3-step SMA(12) forecast. NAIVE copies each burst one interval late; SMA(12) smooths bursts away.

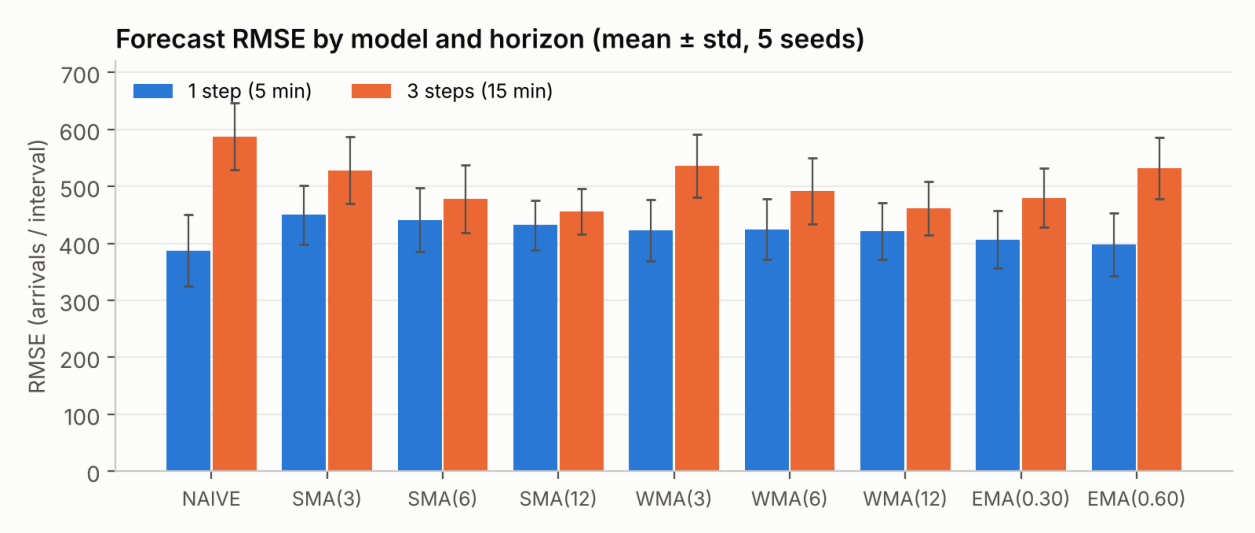


Figure 3. Forecast RMSE by model and horizon, mean and standard deviation over five seeds (274 scored intervals per seed).

Model	MAE h=1	RMSE h=1	MAPE % h=1	MAE h=3	RMSE h=3	MAPE % h=3
NAIVE	178.1	387.4	11.38	314.3	588.0	20.68
SMA(3)	241.2	450.1	15.60	309.5	528.4	20.38
SMA(6)	267.4	441.3	17.39	301.6	478.6	19.98
SMA(12)	278.6	432.4	18.36	299.1	456.6	20.08
WMA(3)	219.0	422.8	14.09	308.3	536.4	20.28
WMA(6)	245.6	425.1	15.87	302.1	492.0	19.89
WMA(12)	263.8	421.7	17.20	297.3	461.4	19.71
EMA(0.30)	238.8	407.0	15.43	297.7	480.2	19.62
EMA(0.60)	206.0	398.1	13.19	304.4	532.2	19.99

Table 1. Module A forecast error, mean over five seeds (arrivals per 5-min interval; mean level about 1,350).

At a 1-step horizon the NAIVE forecast had the lowest RMSE (387) because bursts usually span two intervals, so the last value is a good guess for the next one. At a 3-step horizon the ranking reverses: longer windows (SMA(12), WMA(12)) had the lowest RMSE (about 457-461) and NAIVE the highest (588). No moving average anticipates the start of a burst; all of them only react to it.

6.2 Module B - scaling policies (default configuration)

Policy	Mean wait (s)	p95 latency (s)	SLA viol. %	Thruput (/s)	Worker-h	Cost	Util.	MAPE %
Fixed-12	217.78	577.7 ± 394.2	66.0	5.00	24.1	2.42	0.83	-
Fixed-32	0.11	4.3 ± 0.5	0.0	5.02	64.0	6.42	0.31	-
Reactive	5.34	43.1 ± 33.7	13.2	5.02	45.6	4.58	0.46	-
Predictive	5.50	33.2 ± 15.5	18.7	5.02	31.5	3.17	0.67	21.7
Predictive-CA	5.50	33.2 ± 15.5	18.7	5.02	31.5	3.17	0.67	21.7

Table 2. Module B, default configuration (per-minute billing, 60 s cold start), mean over five seeds.

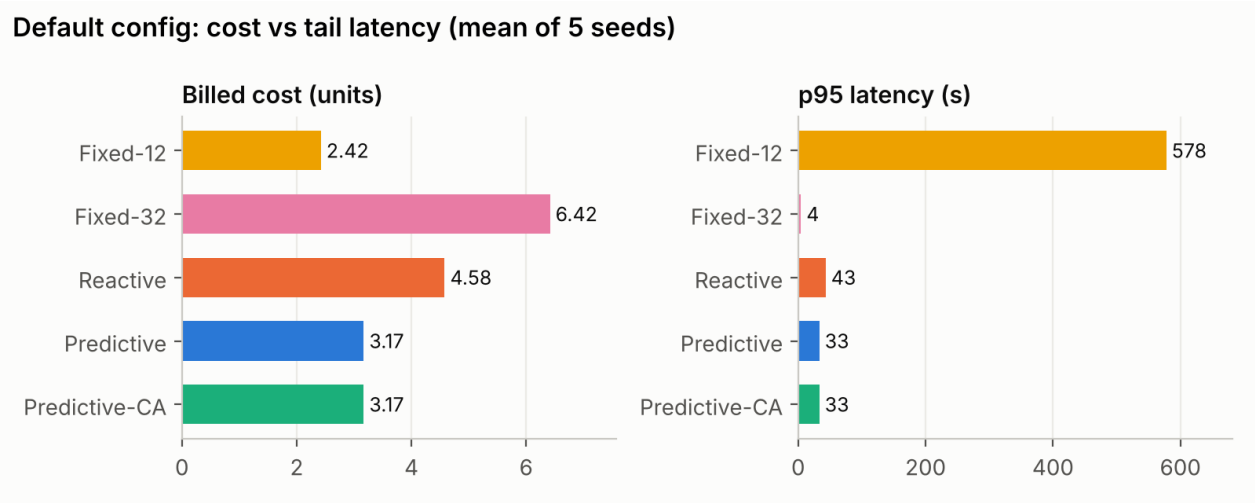


Figure 4. Cost and p95 latency per policy, default configuration. The two fixed allocations bracket the trade-off; the adaptive policies sit between them.

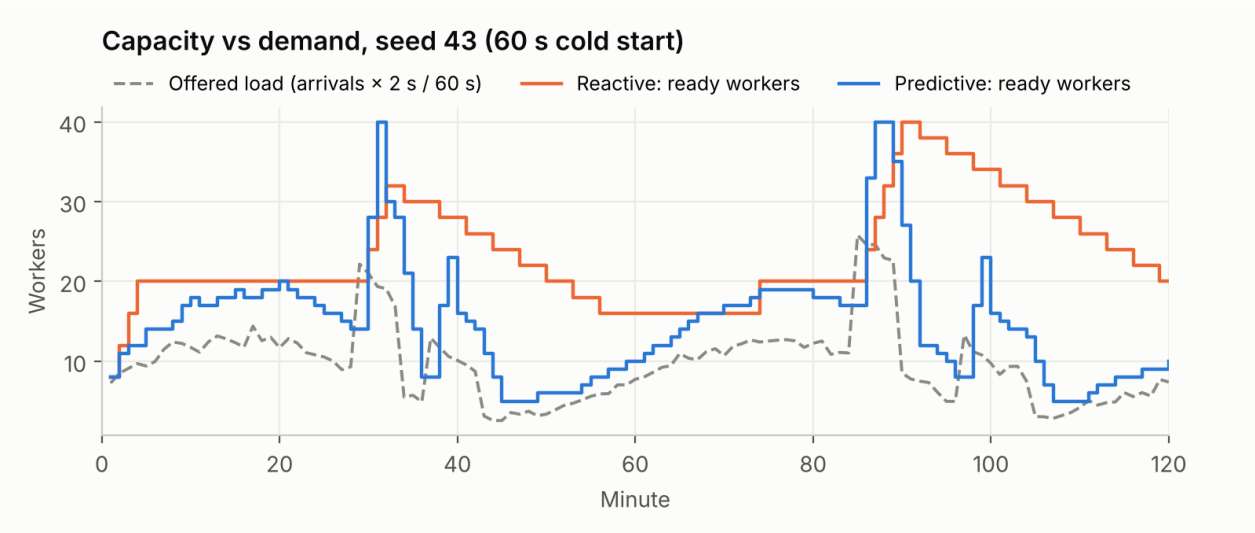


Figure 5. Ready workers over time for the reactive and predictive policies on one workload (seed 43), against the offered load. Reactive scales in slowly and keeps surplus capacity after each peak; predictive follows demand more closely. On this seed, reactive nevertheless had the lower p95 latency (22 s vs 36 s).

Observations for the default configuration (five seeds; spreads are large and no significance test was run):

- Fixed-12 was cheapest (2.42) but queued heavily during peaks (p95 latency 578 s); Fixed-32 almost never queued (p95 4.3 s) but cost 2.6 times as much and ran at 31 % utilization.
- Predictive used 31.5 worker-hours against 45.6 for Reactive (about 31 % fewer), with lower mean p95 latency (33 s vs 43 s) but a higher SLA-violation rate (18.7 % vs 13.2 %). Neither dominates on every metric.
- Predictive-CA was identical to Predictive here. Workers are launched on 60-second ticks and billed in 60-second blocks, so every billing boundary coincides with a tick and no removal is ever deferred.

6.3 Effect of billing granularity

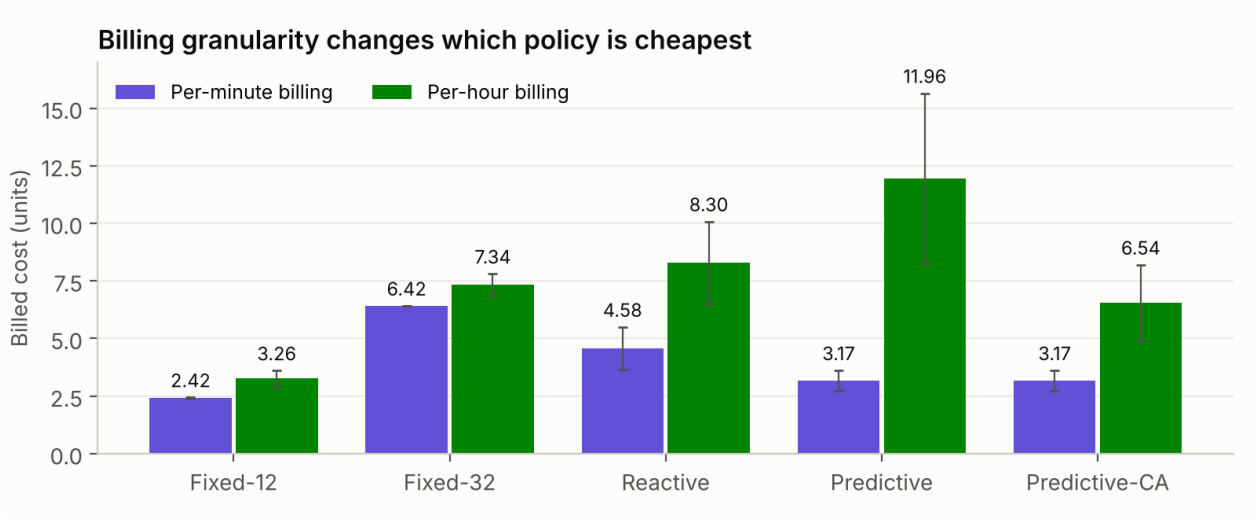


Figure 6. Mean billed cost per policy under per-minute and per-hour billing (same workloads; error bars: std over seeds).

With one-hour billing blocks, plain Predictive became the most expensive policy (11.96): each scale-in followed by a scale-out starts a new paid hour. Predictive-CA reduced the cost to 6.54 (about 45 % lower) and also reduced p95 latency from 33.2 s to 19.8 s, because workers that are already paid for keep serving until their block ends. This is the setting in which the cost-aware mechanism is useful.

6.4 Sensitivity to cold-start delay

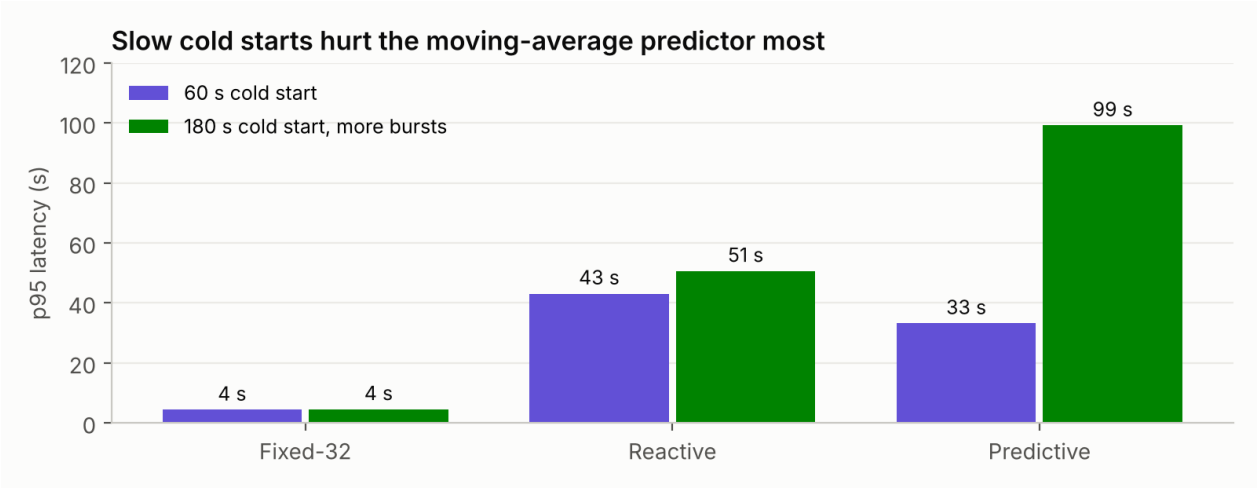


Figure 7. p95 latency with 60 s versus 180 s cold starts (the 180 s configuration also has more frequent bursts).

With 180 s cold starts the predictive policy's p95 latency rose to 99 s, worse than Reactive (51 s). A flat moving-average forecast cannot anticipate a ramp three intervals ahead, so capacity requested at the start of a

burst arrives after it. This negative result is retained deliberately: it identifies the forecaster, not the orchestration mechanism, as the main bottleneck and motivates the next phase.

7. Verification Status

Check	Result
Main and test sources compile (javac --release 17 -Xlint:all -Werror, JDK 21)	Pass
66 unit and integration tests	66 pass, 0 fail (see note)
Hand-computed scenarios (FIFO queue, cold start, cancellation, billing boundaries)	Pass
Conservation laws: integral of queue length = total wait; busy integral = total service	Pass
Re-running an experiment yields byte-identical CSV files	Pass
mvn clean verify	Not yet executed

Note. The development sandbox blocks Maven Central, so Maven could not download its plugins or JUnit. The JUnit 5 test sources were therefore compiled and run against a minimal local re-implementation of the JUnit annotations and assertions, which was itself checked to report failures. Running *mvn clean verify* on a networked machine is the first item of the next phase; its result supersedes this one.

8. Assumptions and Limitations

8.1 Assumptions

- Homogeneous workers; service time independent of worker and load; no failures or interference.
- Single FIFO queue; no priorities, deadlines or task dependencies.
- Perfect, instantaneous monitoring; decisions applied immediately; only worker readiness is delayed.
- Workers present at t = 0 are pre-warmed; cost covers compute time only, in abstract units.
- Workload parameters are illustrative, not fitted to any real system; Module B compresses a daily cycle into one hour.

8.2 Limitations and threats to validity

- **No real-cloud validation.** Absolute values (seconds, cost units) are meaningful only inside the model; only within-configuration comparisons are informative.
- **Synthetic workloads only.** CSV trace replay is implemented but not yet exercised on a public cluster trace.
- **Small samples.** Five seeds, mean and standard deviation only; no confidence intervals or hypothesis tests.
- **Untuned policy parameters.** Thresholds, cooldowns, target utilization and windows were set once, not optimised.
- **Simple forecasters.** Flat moving averages lag ramps and bursts (Section 6.4).
- **End-of-run handling.** Scaling stops when the workload window ends; the remaining backlog is drained by the workers then present, which lengthens the makespan of under-provisioned policies.
- **Research prototype.** It is not production-ready and has not been hardened or security-reviewed.

9. Demonstration Readiness

The prototype can be demonstrated live in about ten minutes: build and test with Maven, run Module A and Module B from the jar, open the per-interval CSV to plot capacity against demand, re-run the hourly-billing configuration to show the cost-aware effect, and re-run any command to show byte-identical output. The README in the repository contains the exact command sequence and expected output.

10. Plan for the Remaining Work (proposed)

Phase	Work item	Expected outcome
1	Run mvn clean verify on a networked machine; set up CI	Authoritative build and test result
2	Trend/seasonality-aware forecasters (Holt, Holt-Winters; optionally ARIMA)	Forecasts that anticipate ramps; re-test the 180 s cold-start case
3	Cost-aware sizing: explicit cost + SLA objective in the predictive policy	Policy that trades cost and latency by a tunable weight
4	Replay aggregated public cluster traces through both modules	Results on realistic arrival patterns
5	Parameter sweeps, 20-30 seeds, confidence intervals and paired tests	Statistically supported comparisons
6	Optional small-scale check on local containers (e.g. Docker/Kubernetes)	Sanity check of simulator assumptions
7	Thesis writing and final evaluation	Final report and presentation

References

[1] T. Lorido-Botran, J. Miguel-Alonso and J. A. Lozano, "A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments," *Journal of Grid Computing*, vol. 12, no. 4, pp. 559-592, 2014.

[2] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose and R. Buyya, "CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Software: Practice and Experience*, vol. 41, no. 1, pp. 23-50, 2011.

[3] P. A. W. Lewis and G. S. Shedler, "Simulation of nonhomogeneous Poisson processes by thinning," *Naval Research Logistics Quarterly*, vol. 26, no. 3, pp. 403-413, 1979.

[4] J. D. C. Little, "A Proof for the Queuing Formula: L = lambda W," *Operations Research*, vol. 9, no. 3, pp. 383-387, 1961.

[5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, 2021.

[6] N. Roy, A. Dubey and A. Gokhale, "Efficient Autoscaling in the Cloud Using Predictive Models for Workload Forecasting," in *Proc. IEEE 4th International Conference on Cloud Computing (CLOUD)*, 2011, pp. 500-507.

Appendix: all figures and tables were generated from the prototype's own CSV output (results/*/orchestration_aggregate.csv, forecast_aggregate.csv and per-interval series) for seeds 42-46.